PROPOSAL PROYEK

Pengembangan Task Manager Sederhana

dengan JavaScript (Node.js) menggunakan AI Generatif sebagai Asisten Pemrograman

Analisis - Perancangan - Prompting - Evaluasi Kode - Pengujian - Debugging - Perbaikan

Keterangan	Isi
Nama	[isi nama Anda]
NIM / Kelas	[isi NIM / kelas]
Mata kuliah / Dosen	[isi mata kuliah dan dosen]
Repositori GitHub	https://github.com/[username]/task-manager-js
Tanggal	18 Oktober 2026

Seluruh hasil pengujian, tangkapan layar, dan riwayat perubahan dalam dokumen ini berasal dari proyek yang benar-benar dijalankan dan tersimpan di repositori.

Daftar Isi

1. Pendahuluan
2. Analisis Kebutuhan
3. Perancangan Program
4. Strategi Penggunaan AI Generatif (Prompting)
5. Menulis Kode dan Mengevaluasi Hasil AI
6. Pengujian dan Identifikasi Kesalahan
7. Debugging dan Perbaikan Kode
8. Hasil Akhir dan Cara Menjalankan
9. Refleksi dan Penjelasan Keputusan
10. Penutup dan Lampiran

1. Pendahuluan

1.1 Latar Belakang

Pengelolaan tugas harian sering berantakan ketika hanya mengandalkan ingatan atau catatan acak. Aplikasi task manager membantu mencatat tugas, menetapkan tenggat, dan menandai progres. Pada saat yang sama, AI generatif seperti Claude, ChatGPT, atau Copilot kini mampu menghasilkan kode dengan cepat. Namun kode hasil AI **tidak otomatis benar**: ia bisa mengandung bug halus, melewatkan kasus tepi, atau terlihat meyakinkan padahal keliru.

Proyek ini membuat task manager sederhana berbasis JavaScript sekaligus mendemonstrasikan cara bekerja dengan AI secara bertanggung jawab: **merancang** sebelum menulis, **memberi instruksi** (prompt) yang spesifik, **mengevaluasi** kode yang dihasilkan secara kritis, **menguji**, **mengidentifikasi kesalahan**, **memperbaiki**, dan **menjelaskan alasan** setiap keputusan. AI diperlakukan sebagai asisten, bukan sebagai pengganti pemahaman programmer.

1.2 Tujuan

- Membangun aplikasi task manager CLI dengan JavaScript (Node.js) yang berfungsi dan teruji.
- Menunjukkan alur kerja pemrograman dengan AI: rancang, tulis, uji, debug, perbaiki.
- Mendokumentasikan contoh prompt beserta alasannya, serta cara menilai dan memperbaiki keluaran AI.
- Menyimpan seluruh hasil (kode, cara menjalankan, inisialisasi, konfigurasi, proposal) di GitHub.

1.3 Ruang Lingkup dan Batasan

- **Termasuk:** tambah, lihat (dengan filter), selesaikan, dan hapus tugas; tenggat opsional; penyimpanan JSON; konfigurasi; tes otomatis.
- **Tidak termasuk:** antarmuka grafis/web, akun pengguna multi-user, sinkronisasi cloud, notifikasi.
- **Batasan teknis:** hanya Node.js bawaan (tanpa library eksternal) agar mudah dijalankan dan dipahami.

1.4 Manfaat

Bagi pembelajar, dokumen ini menjadi contoh nyata bahwa kualitas hasil AI sangat bergantung pada kualitas instruksi dan kedisiplinan verifikasi. Bagi penilai, setiap klaim dapat ditelusuri ke bukti: keluaran tes, riwayat commit git, dan kode di repositori.

2. Analisis Kebutuhan

2.1 Kebutuhan Fungsional

Kode	Kebutuhan	Kriteria penerimaan
FR-01	Menambah tugas dengan judul dan tenggat opsional	Tugas tersimpan dengan ID unik; judul kosong ditolak; tanggal harus valid (YYYY-MM-DD)
FR-02	Menampilkan daftar tugas	Terurut menurut tenggat terdekat; tugas tanpa tenggat di bagian akhir; bisa difilter all/pending/done

Kode	Kebutuhan	Kriteria penerimaan
FR-03	Menandai tugas selesai	Status berubah menjadi selesai; ID tidak ada menghasilkan pesan error jelas
FR-04	Menghapus tugas	Tugas hilang dari daftar; ID baru tidak bentrok dengan ID lama
FR-05	Menandai tugas terlambat	Tugas belum selesai dengan tenggat lampau diberi label TERLAMBAT
FR-06	Menyimpan data permanen	Data tetap ada setelah program ditutup; file rusak tidak menyebabkan crash

2.2 Kebutuhan Non-Fungsional

- **Kemudahan dijalankan:** tanpa instalasi dependensi; satu perintah `node bin/tm.js`.
- **Keterujian (testability):** logika bisnis terpisah dari input/output agar dapat diuji otomatis.
- **Ketahanan:** input salah dan file data rusak ditangani dengan pesan jelas, bukan crash.
- **Keterbacaan:** kode modular dengan komentar singkat berbahasa Indonesia.
- **Portabilitas:** berjalan di Windows, macOS, dan Linux (Node.js 20+).

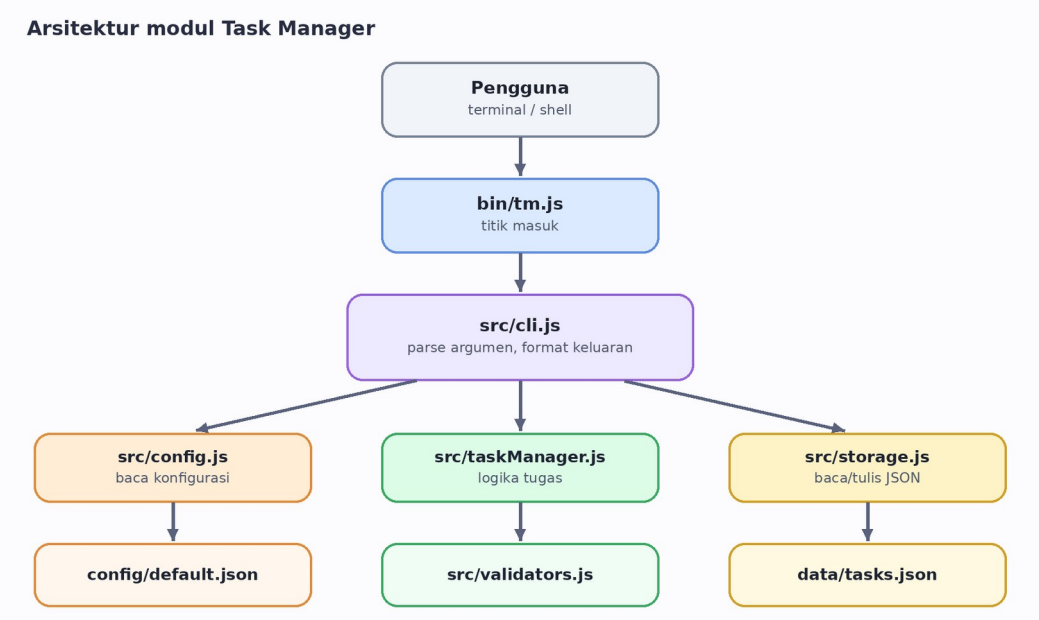
2.3 Pemilihan Teknologi dan Alasannya

Pilihan	Alternatif	Alasan memilih
Node.js (CLI)	Aplikasi web / React	Fokus proyek adalah proses dengan AI, bukan UI. CLI lebih kecil sehingga setiap baris kode AI bisa direview penuh.
Penyimpanan file JSON	SQLite / database	Sederhana, mudah dibaca manusia, tanpa dependensi. Cukup untuk ratusan tugas.
`node:test` bawaan	Jest / Mocha	Sudah ada di Node; menghilangkan instalasi library dan menyederhanakan CI.
Tanpa library parser argumen	commander / yargs	Perintahnya hanya 6; parser kecil (±15 baris) mudah diuji dan dipahami.

3. Perancangan Program

3.1 Arsitektur Modul

Program dibagi menjadi modul dengan satu tanggung jawab masing-masing. Pemisahan terpenting: **logika tugas (`taskManager.js`) tidak boleh menyentuh file atau terminal.** Dengan begitu logika dapat diuji cepat tanpa file sungguhan, dan penyimpanan dapat diganti tanpa mengubah logika.



Gambar 1. Arsitektur modul Task Manager: CLI memanggil config, logika tugas, dan penyimpanan.

3.2 Struktur Data

Field	Tipe	Keterangan
id	number	ID terbesar yang ada + 1 (unik, tidak bentrok setelah penghapusan)
title	string	Judul, di-trim, tidak boleh kosong
due	string | null	Tenggat format YYYY-MM-DD, atau null jika tidak ada
done	boolean	Status selesai
createdAt	string (ISO 8601)	Waktu pembuatan

3.3 Alur Perintah

1. `bin/tm.js` meneruskan argumen terminal ke `run()` di `cli.js`.
2. `cli.js` mem-parsing argumen dan memuat konfigurasi (env mengalahkan `config/default.json`).
3. Data dibaca lewat `storage.loadTasks()` lalu dibungkus `TaskManager`.
4. Perintah dijalankan; jika mengubah data, hasil disimpan lewat `storage.saveTasks()` (tulis ke file sementara lalu rename agar atomik).
5. Error apa pun ditangkap satu tempat: pesan ke stderr dan exit code 1.

3.4 Keputusan Rancangan Penting

Keputusan	Alasan	Konsekuensi / trade-off
`run(argv, {out, err, env})` menerima dependensi sebagai parameter	Agar CLI bisa diuji tanpa terminal sungguhan dan tanpa menyentuh data pengguna	Sedikit lebih verbose, tetapi tes end-to-end jadi mudah

Keputusan	Alasan	Konsekuensi / trade-off
ID = max + 1, bukan counter terpisah	Menjaga format file tetap berupa array sederhana	ID tugas terakhir yang dihapus dapat dipakai ulang; dapat diterima untuk skala proyek ini
Tugas tanpa tenggat selalu di akhir	Tugas bertenggat lebih mendesak untuk dilihat pengguna	Perlu pembanding khusus saat mengurutkan
File rusak dibackup ke `.bak`, bukan ditimpa	Data pengguna tidak boleh hilang diam-diam	Ada file tambahan yang perlu dibersihkan manual

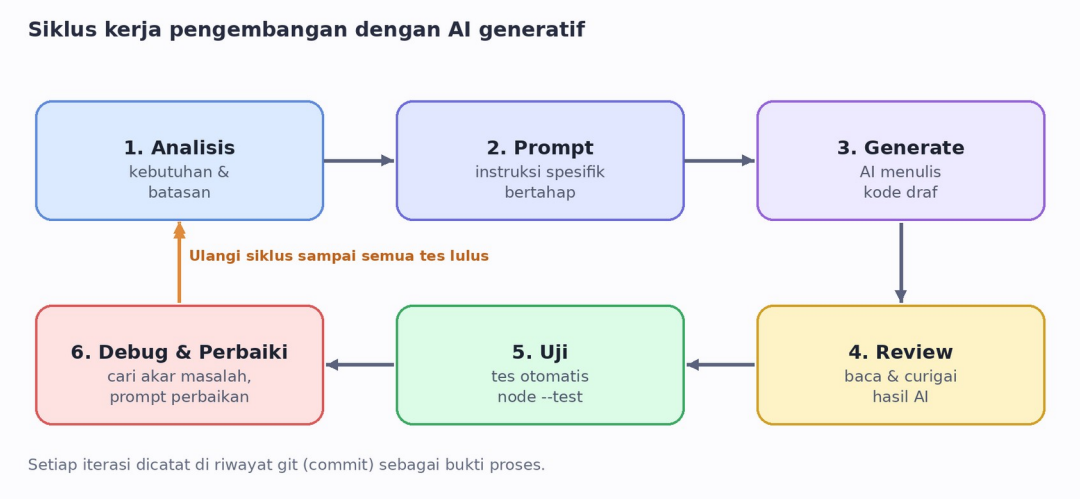
3.5 Rencana Pengujian

Sebelum meminta kode, ditetapkan bahwa tes harus mencakup (a) jalur normal, (b) kasus tepi: ID setelah penghapusan, ID tidak ada, judul kosong, tanggal mustahil, tugas tanpa tenggat, file belum ada, file JSON rusak, dan (c) alur end-to-end lewat CLI. Tes ditulis dengan `node:test` dan dijalankan dengan `npm test`.

4. Strategi Penggunaan AI Generatif (Prompting)

4.1 Siklus Kerja

Pekerjaan dengan AI diperlakukan sebagai siklus berulang, bukan sekali tanya. Setiap putaran selesai hanya ketika tes otomatis lulus.



Gambar 2. Siklus kerja: analisis, prompt, generate, review, uji, debug & perbaiki, lalu ulangi.

4.2 Prinsip Menyusun Instruksi

Prinsip	Penerapan di proyek ini	Alasan
Berikan peran dan konteks	"Kamu senior JavaScript developer... saya mahasiswa..."	Menyesuaikan kedalaman dan gaya penjelasan
Nyatakan batasan	"Tanpa library eksternal", "kelas tidak boleh I/O"	Mencegah solusi berlebihan dan menjaga desain bersih

Prinsip	Penerapan di proyek ini	Alasan
Minta rancangan sebelum kode	Prompt 1 meminta struktur modul + alasan	Menangkap kesalahan arsitektur lebih murah daripada kesalahan kode
Satu tugas per instruksi	Prompt 2 hanya untuk `taskManager.js`	Hasil kecil mudah direview dan di-debug
Sebut kasus tepi secara eksplisit	Prompt 3 mendaftar kasus tepi	AI cenderung menulis tes jalur bahagia saja
Sertakan bukti saat debugging	Prompt 4 menempel kode + pesan error asli	Menghindari tebakan; fokus pada akar masalah

4.3 Contoh Instruksi, Gambar, dan Alasannya

Empat prompt berikut mewakili empat fase kerja. Di bawah tiap kartu dijelaskan **mengapa** instruksi dirumuskan demikian.

Prompt 1 - Rancang (peran + konteks + batasan + format)

Kamu adalah senior JavaScript developer. Saya mahasiswa dan ingin membuat task manager CLI sederhana dengan Node.js tanpa library eksternal.
Fitur: tambah tugas (judul, tenggat opsional), daftar tugas (filter status), tandai selesai, hapus. Data disimpan di file JSON.
Sebelum menulis kode, usulkan struktur file dan pembagian modul (logika, penyimpanan, CLI). Jelaskan alasan tiap keputusan dalam bahasa Indonesia.

Alasan instruksi ini:
Meminta rancangan sebelum kode mencegah AI langsung menulis satu file raksasa. Peran memberi gaya jawaban yang tepat, batasan 'tanpa library' menjaga proyek tetap sederhana, dan permintaan alasan membuat saya bisa menilai keputusan AI, bukan sekadar menyalinnya.

Gambar 3. Prompt 1 (merancang): meminta struktur dan alasan sebelum kode.

Prompt 2 - Tulis kode (satu modul per instruksi)

Tulis src/taskManager.js: class TaskManager dengan add, complete, remove, list(status). Kelas ini tidak boleh membaca/menulis file atau memakai console.
Setiap tugas: id, title, due, done, createdAt. Beri komentar singkat berbahasa Indonesia.

Alasan instruksi ini:
Instruksi kecil dan satu tujuan membuat hasil mudah direview. Larangan I/O di dalam kelas memaksa pemisahan tanggung jawab sehingga logika bisa diuji tanpa file nyata.

Gambar 4. Prompt 2 (menulis): satu modul, dengan larangan I/O di logika.

Prompt 3 - Uji: minta AI menyerang kodenya sendiri

Tulis tes dengan node:test untuk TaskManager dan storage. Fokus pada kasus tepi: ID setelah penghapusan, ID yang tidak ada, judul kosong, tanggal tidak valid (2026-13-45), tugas tanpa tenggat, file belum ada, file JSON rusak.
Jangan sesuaikan tes agar lulus dengan kode yang ada; tes harus menggambarkan perilaku yang benar.

Alasan instruksi ini:

AI cenderung menulis tes 'jalur bahagia' yang pasti lulus. Menyebut kasus tepi dan melarang tes disesuaikan dengan kode membuat tes menjadi alat deteksi bug yang jujur.

Gambar 5. Prompt 3 (menguji): AI diminta menyerang kodenya sendiri lewat kasus tepi.

Prompt 4 - Debug: beri bukti, bukan keluhan

Tes ini gagal. Kode: [tempel taskManager.js]. Output: 'AssertionError: ID duplikat: 2,2'.
Jelaskan akar masalahnya (bukan hanya gejalanya), beri perbaikan minimal, lalu sebutkan tes lain yang mungkin terdampak.

Alasan instruksi ini:

Menempelkan kode + pesan error asli menghindari tebakan. Meminta 'akar masalah' dan 'perbaikan minimal' mencegah AI menulis ulang seluruh file dan menimbulkan bug baru.

Gambar 6. Prompt 4 (debugging): menyertakan kode dan pesan error asli.

4.4 Kriteria Mengevaluasi Kode dari AI

Kode AI tidak pernah diterima begitu saja. Setiap hasil dinilai dengan daftar periksa berikut sebelum dan sesudah dijalankan:

Pertanyaan evaluasi	Cara memeriksa
Apakah memenuhi seluruh kebutuhan (FR-01 s.d. FR-06)?	Cocokkan fitur dengan tabel kebutuhan
Apa yang terjadi jika input kosong, salah, atau tidak ada?	Telusuri tiap fungsi dengan input ekstrem; tulis tesnya
Adakah asumsi tersembunyi (mis. data selalu valid, file selalu ada)?	Cari akses properti tanpa pengecekan, I/O tanpa try/catch
Apakah tanggung jawab modul bersih?	Logika tidak boleh memanggil console atau fs
Apakah tes benar-benar membuktikan sesuatu?	Perkuat tes; curigai tes yang lulus terlalu mudah
Bisakah saya menjelaskan setiap baris?	Jika tidak, minta AI menjelaskan atau sederhanakan

5. Menulis Kode dan Mengevaluasi Hasil AI

5.1 Draf Awal (v1)

Draf pertama dari AI terdiri dari `taskManager.js` (±35 baris) dan `storage.js` (±12 baris). Secara sekilas kodenya rapi, mudah dibaca, dan "terlihat benar". Inilah jebakannya: **keterbacaan bukan bukti kebenaran.** Berikut cuplikan kritis dari v1:

```
add(title, due) {
  const task = { id: this.tasks.length + 1, title, due, done: false, ... };
}
complete(id) {
  const task = this.tasks.find((t) => t.id === id);
  task.done = true;                       // jika task undefined -> crash
}
list(status) { ... .sort((a, b) => a.due.localeCompare(b.due)); }  // due
bisa kosong
loadTasks(file) { return JSON.parse(fs.readFileSync(file, 'utf8')); }  //
file ada & valid?
```

5.2 Hasil Review Manual (sebelum dijalankan)

Membaca v1 dengan daftar periksa pada bagian 4.4 menghasilkan lima kecurigaan. Semuanya kemudian dibuktikan atau dibantah oleh tes pada bab 6:

No	Kecurigaan saat membaca kode	Prediksi
1	`id = panjang array + 1` akan bentrok setelah ada penghapusan	ID duplikat
2	`find()` bisa mengembalikan `undefined`, lalu langsung diakses	Crash TypeError
3	`a.due.localeCompare` jika `due` kosong	Crash TypeError
4	`readFileSync` / `JSON.parse` tanpa penanganan error	Crash pada pengguna baru / file rusak
5	Tidak ada validasi judul dan tanggal	Data sampah tersimpan

Poin positif v1 yang dipertahankan: pemisahan kelas dan penyimpanan, nama fungsi jelas, penggunaan `filter` yang immutable, dan penyalinan array sebelum `sort` (`[...result]`) sehingga data asli tidak berubah.

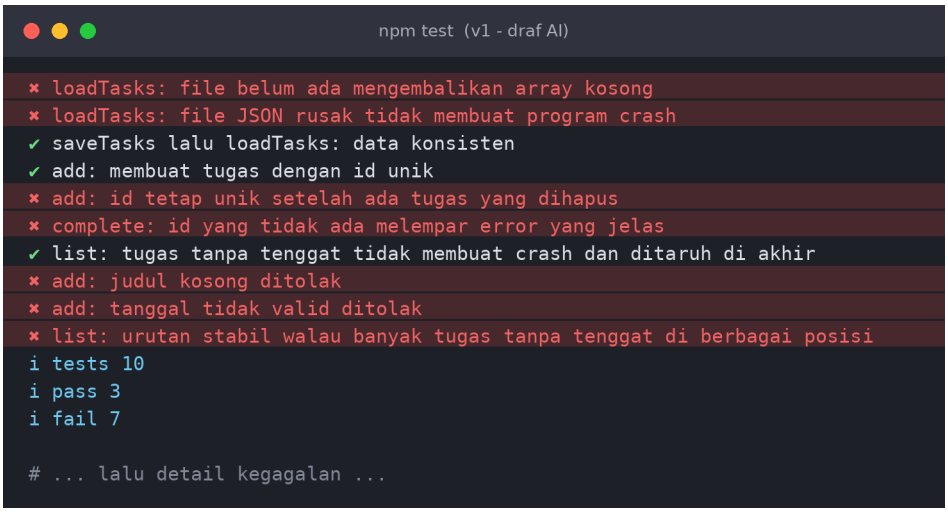
6. Pengujian dan Identifikasi Kesalahan

6.1 Kendala Pertama: Perintah Tes Gagal Dijalankan

Sebelum tes dapat berjalan, muncul kesalahan pada konfigurasi yang diusulkan AI: `"test": "node --test test/"`. Pada Node 22 perintah itu dianggap sebagai path modul dan berhenti dengan pesan **`Could not find 'test/'`**. Perbaikannya: gunakan penemuan otomatis `node --test` (ditambah

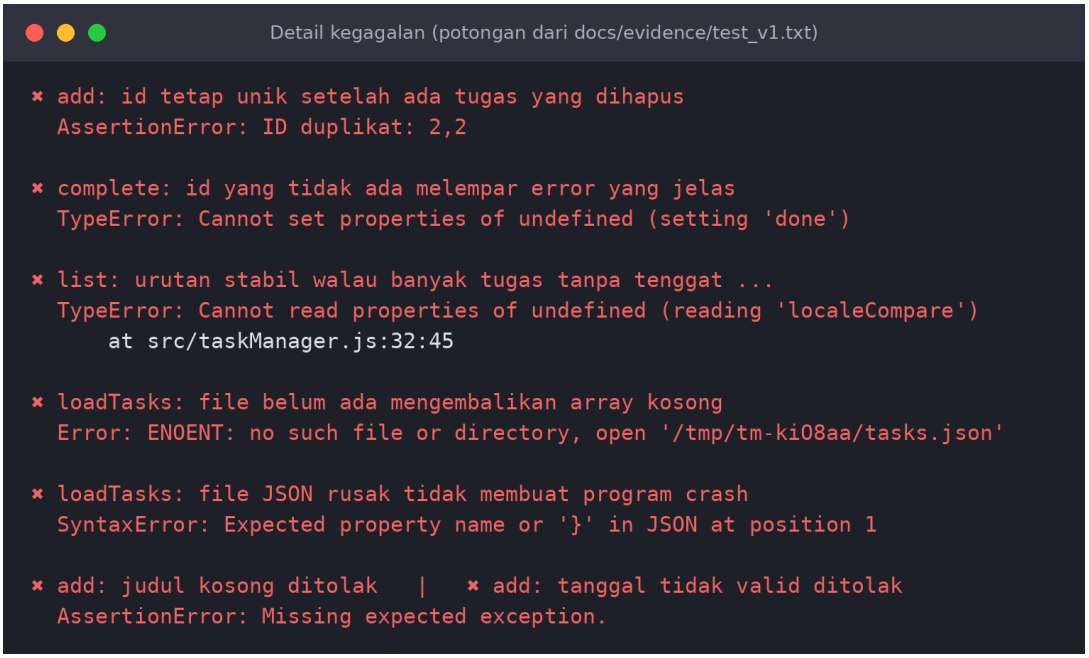
reporter `spec` agar hasil mudah dibaca). Pelajaran: bahkan skrip konfigurasi dari AI harus dijalankan, bukan dipercaya.

6.2 Hasil Tes pada v1



Gambar 7. Hasil npm test pada draf AI v1: 7 dari 10 tes gagal (baris merah).

Tujuh kegagalan menunjuk ke bug nyata. Detail pesan error asli ditunjukkan pada gambar berikut.



Gambar 8. Pesan error asli dari tiap tes yang gagal.

6.3 Pemetaan Gejala ke Akar Masalah

Bug	Gejala di tes	Akar masalah	Dampak bagi pengguna
B1	ID duplikat: 2,2	ID dihitung dari `length + 1`	Menyelesaikan/menghapus tugas yang salah
B2	TypeError ... setting 'done'	`find()` tidak dicek; `remove` diam-diam tidak melakukan apa pun	Crash dengan pesan teknis membingungkan

Bug	Gejala di tes	Akar masalah	Dampak bagi pengguna
B3	TypeError ... 'localeCompare'	Pembanding mengasumsikan `due` selalu string	Crash saat ada tugas tanpa tenggat
B4	ENOENT dan SyntaxError	Tidak menangani file belum ada / JSON rusak	Pengguna baru langsung crash; data bisa hilang
B5	Missing expected exception (x2)	Tidak ada validasi judul & tanggal	Tugas tanpa judul & tanggal 2026-13-45 tersimpan

6.4 Pelajaran: Tes yang Lulus Belum Tentu Benar

Pada draf pertama, tes "tugas tanpa tenggat tidak membuat crash" **lulus**, padahal bug B3 ada. Penyebabnya kebetulan: urutan pemanggilan pembanding oleh mesin JavaScript untuk array dua elemen membuat `due` yang kosong hanya berada di sisi argumen kedua, sehingga `localeCompare(undefined)` (yang membandingkan dengan teks "undefined") kebetulan memberi urutan yang benar. Ketika tes diperkuat dengan empat tugas (dua tanpa tenggat di posisi berbeda), kode langsung crash. Kesimpulan: **tes harus dirancang untuk mencoba menggagalkan kode**, bukan sekadar membuatnya hijau.

7. Debugging dan Perbaikan Kode

Setiap perbaikan dibuat **minimal** dan terarah ke akar masalah, lalu seluruh tes dijalankan ulang untuk memastikan tidak ada regresi. Gambar di bawah menunjukkan perubahan kode (merah = dihapus, hijau = ditambah).

7.1 B1 - ID Duplikat

Diagnosis: setelah menghapus tugas #1 dari [#1, #2], panjang array menjadi 1 sehingga tugas baru mendapat ID 2, sama dengan tugas yang sudah ada. **Keputusan:** hitung ID dari ID terbesar + 1. Alternatif UUID ditolak karena pengguna harus mengetik ID di terminal, jadi angka pendek lebih ramah.

Perbaikan Bug #1 - ID duplikat (src/taskManager.js)

```
-     id: this.tasks.length + 1,
+     id: this.nextId(),   // ID terbesar + 1
+
+   nextId() {
+     return this.tasks.reduce((max, t) => Math.max(max, t.id), 0) + 1;
+   }
```

Gambar 9. Perbaikan B1: ID dari nilai maksimum + 1.

7.2 B2 - Tugas Tidak Ditemukan

Diagnosis: `find()` mengembalikan `undefined` bila ID tidak ada, lalu `task.done = true` melempar TypeError. **Keputusan:** satu metode `get()` yang dipakai bersama oleh `complete` dan `remove` dan melempar pesan berbahasa pengguna. `Number(id)` dipakai karena argumen CLI selalu berupa string.

Perbaikan Bug #2 - tugas tidak ditemukan (src/taskManager.js)

```
-     const task = this.tasks.find((t) => t.id === id);
-     task.done = true;
+     const task = this.get(id);   // melempar error jelas jika tidak ada
+     task.done = true;
+
+   get(id) {
+     const task = this.tasks.find((t) => t.id === Number(id));
+     if (!task) throw new Error(`Tugas dengan ID ${id} tidak ditemukan`);
+     return task;
+   }
```

Gambar 10. Perbaikan B2: metode get() dengan pesan error yang jelas.

7.3 B3 - Pengurutan Tanpa Tenggat

Diagnosis: pembanding memanggil `.localeCompare` pada nilai kosong. **Keputusan:** pembanding eksplisit tiga cabang - tenggat sama dibandingkan ID (urutan stabil), kosong ke akhir, selain itu bandingkan teks tanggal. Format ISO `YYYY-MM-DD` sengaja dipilih karena urutan teksnya sama dengan urutan kronologis.

Perbaikan Bug #3 - pengurutan tanpa tenggat (src/taskManager.js)

```
-     return [...result].sort((a, b) => a.due.localeCompare(b.due));
+     return [...result].sort((a, b) => {
+       if (a.due === b.due) return a.id - b.id;
+       if (a.due === null || a.due === undefined) return 1;
+       if (b.due === null || b.due === undefined) return -1;
+       return a.due < b.due ? -1 : 1;
+     });
```

Gambar 11. Perbaikan B3: pembanding yang aman terhadap nilai kosong.

7.4 B4 dan B5 - Penyimpanan dan Validasi

Penyimpanan: `ENOENT` dianggap kondisi normal (pengguna baru) sehingga mengembalikan array kosong; JSON rusak disalin ke `.bak` sebelum mulai dari kosong sehingga data tidak hilang; penulisan memakai file sementara + rename agar file tidak pernah setengah tertulis.

```
try { raw = fs.readFileSync(file, 'utf8'); }
catch (err) { if (err.code === 'ENOENT') return []; throw err; }
try { const data = JSON.parse(raw); if (!Array.isArray(data)) throw ...;
return data; }
catch { fs.copyFileSync(file, `${file}.bak`); return []; }
```

Validasi: modul `validators.js` baru. Tanggal tidak cukup dicek dengan regex karena `2026-13-45` lolos regex; kode membuat objek `Date` UTC lalu memastikan tahun, bulan, dan hari tetap sama (round-trip) sehingga tanggal yang tidak ada di kalender ditolak.

7.5 B6 - Ditemukan oleh Tes End-to-End

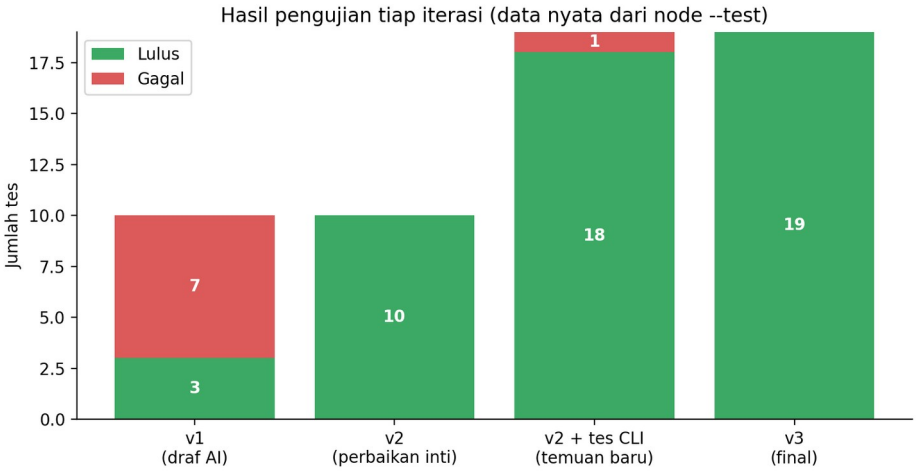
Setelah v2 lulus 10 tes inti, ditulis 9 tes CLI end-to-end. Satu gagal: `add "Tugas" --due` (tanpa nilai) **diam-diam menyimpan tugas tanpa tenggat**. Pengguna mengira tenggat tersimpan. Bug ini tidak terlihat oleh tes unit karena berada di lapisan parsing argumen. **Keputusan:** parser melempar error "Opsi --due membutuhkan nilai" - lebih baik gagal dengan suara keras daripada diam-diam salah.

Perbaikan Bug #6 - opsi tanpa nilai (src/cli.js)

```
-       out[argv[i].slice(2)] = argv[i + 1];
-       i++;
+       const value = argv[i + 1];
+       if (value === undefined || value.startsWith('--')) {
+         throw new Error(`Opsi ${argv[i]} membutuhkan nilai`);
+       }
+       out[argv[i].slice(2)] = value;
+       i++;
```

Gambar 12. Perbaikan B6: opsi tanpa nilai ditolak.

7.6 Ringkasan Hasil Tiap Iterasi

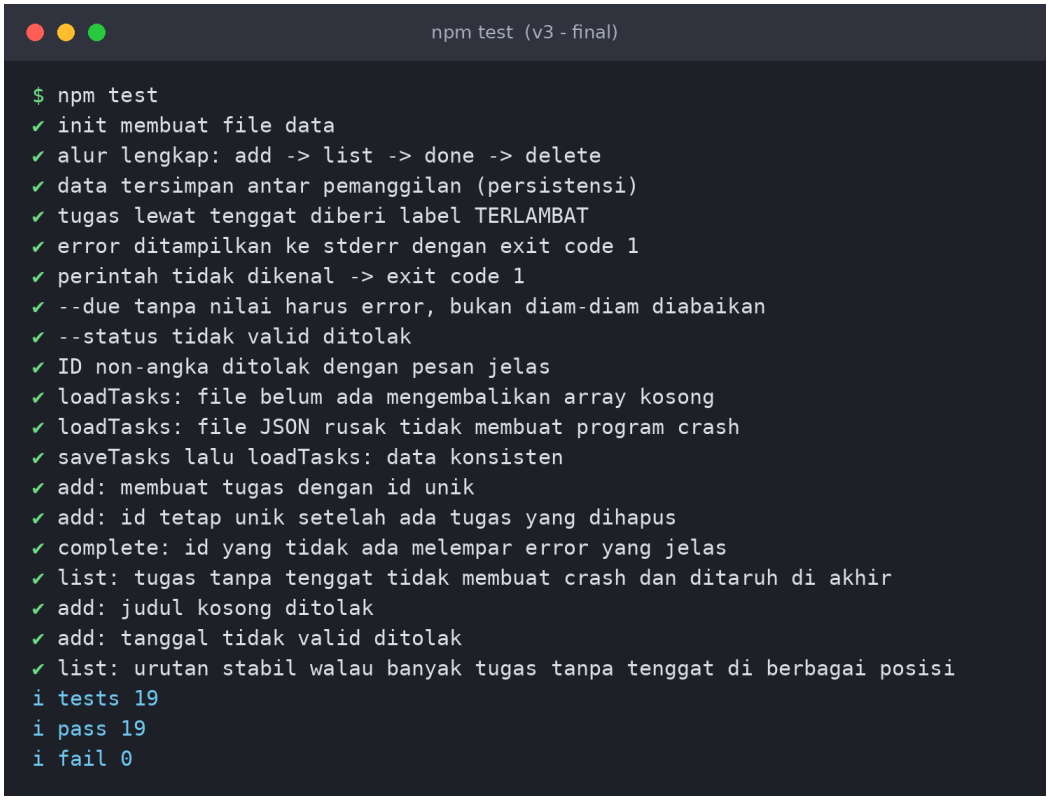


Gambar 13. Jumlah tes lulus dan gagal pada tiap iterasi (data dari keluaran node --test).

Iterasi	Perubahan utama	Total tes	Lulus	Gagal
v1	Draf awal dari AI	10	3	7
v2	Perbaikan B1-B5, tambah validators, config, CLI	10	10	0
v2 + tes CLI	Ditambah 9 tes end-to-end	19	18	1 (B6)
v3	Perbaikan B6 pada parser argumen	19	19	0

8. Hasil Akhir dan Cara Menjalankan

8.1 Hasil Pengujian Akhir



Gambar 14. Hasil akhir npm test: 19 tes, 19 lulus, 0 gagal.

8.2 Demonstrasi Penggunaan

Gambar berikut adalah keluaran nyata program: inisialisasi, penambahan empat tugas, penyelesaian tugas #1, daftar dengan label TERLAMBAT (tenggat 1 Okt 2026 sudah lewat), penghapusan, dan filter pending. Perhatikan tugas tanpa tenggat berada di akhir daftar.

Contoh penggunaan Task Manager

```
$ node bin/tm.js init
Siap. File data: /tmp/demo/tasks.json
$ node bin/tm.js add "Kerjakan proposal" --due 2026-10-25
Tugas #1 ditambahkan: Kerjakan proposal
$ node bin/tm.js add "Push kode ke GitHub" --due 2026-10-28
Tugas #2 ditambahkan: Push kode ke GitHub
$ node bin/tm.js add "Baca dokumentasi Node.js" --due 2026-10-01
Tugas #3 ditambahkan: Baca dokumentasi Node.js
$ node bin/tm.js add "Ide fitur baru"
Tugas #4 ditambahkan: Ide fitur baru
$ node bin/tm.js done 1
Tugas #1 selesai: Kerjakan proposal
$ node bin/tm.js list
  3  [ ] Baca dokumentasi Node.js (tenggat 2026-10-01) TERLAMBAT
  1  [x] Kerjakan proposal (tenggat 2026-10-25)
  2  [ ] Push kode ke GitHub (tenggat 2026-10-28)
  4  [ ] Ide fitur baru (tanpa tenggat)
$ node bin/tm.js delete 4
Tugas #4 dihapus: Ide fitur baru
$ node bin/tm.js list --status pending
  3  [ ] Baca dokumentasi Node.js (tenggat 2026-10-01) TERLAMBAT
  2  [ ] Push kode ke GitHub (tenggat 2026-10-28)
```

Gambar 15. Contoh sesi penggunaan Task Manager.

8.3 Penanganan Error

Penanganan error yang ramah pengguna

```
$ node bin/tm.js done 99
Error: Tugas dengan ID 99 tidak ditemukan
$ node bin/tm.js add "Rapat" --due 2026-13-45
Error: Tanggal tidak ada di kalender: "2026-13-45"
$ node bin/tm.js add "Rapat" --due
Error: Opsi --due membutuhkan nilai
$ node bin/tm.js add "   "
Error: Judul tugas tidak boleh kosong
$ node bin/tm.js list --status selesai
Error: Status tidak dikenal: "selesai"
```

Gambar 16. Pesan error yang ramah untuk berbagai input salah.

8.4 Cara Menjalankan (ringkas)

```
git clone https://github.com/<username>/task-manager-js.git
cd task-manager-js
npm run init                                  # inisialisasi file data
node bin/tm.js add "Belajar JS" --due 2026-11-01
node bin/tm.js list
npm test                                      # jalankan 19 tes
```

Konfigurasi: lokasi file data dan warna diatur di `config/default.json`, dan dapat ditimpa variabel lingkungan `TM_DATA_FILE` dan `NO_COLOR`. Panduan lengkap ada di `README.md` pada repositori.

9. Refleksi dan Penjelasan Keputusan

9.1 Apa yang Dilakukan AI dengan Baik

- Menghasilkan kerangka kode dan struktur modul dengan cepat dan rapi.
- Menulis tes dan pesan error dalam jumlah besar saat diberi daftar kasus tepi yang jelas.
- Menjelaskan akar masalah ketika diberi kode dan pesan error asli.

9.2 Kelemahan Kode AI yang Ditemukan

- **Optimis berlebihan:** menganggap input selalu valid, file selalu ada, dan data selalu lengkap.
- **Gagal pada kasus tepi** yang tidak disebut eksplisit (ID setelah penghapusan, nilai kosong).
- **Konfigurasi yang tidak dijalankan:** skrip `npm test` gagal pada Node 22.
- **Percaya diri tanpa bukti:** kode terlihat benar padahal 7 dari 10 tes gagal.

9.3 Ringkasan Keputusan dan Alasannya

Keputusan	Alasan
Merancang sebelum meminta kode	Kesalahan arsitektur paling mahal diperbaiki belakangan
Memisahkan logika dari I/O	Memungkinkan tes cepat dan penggantian penyimpanan
Menulis tes sebelum memperbaiki	Bug terbukti nyata, dan perbaikan terukur
Perbaikan minimal per bug	Mengurangi risiko regresi dan memudahkan review
Gagal keras (error jelas) dibanding diam-diam	Pengguna tahu apa yang salah; data tidak rusak senyap
Satu commit per iterasi	Riwayat git menjadi bukti proses yang bisa diaudit
Backup file rusak, bukan menimpa	Data pengguna adalah aset; jangan hilang diam-diam

9.4 Etika dan Tanggung Jawab

Penggunaan AI diungkapkan secara terbuka dalam dokumen dan repositori. Tanggung jawab atas kebenaran kode tetap pada pengembang: setiap bagian dijalankan, diuji, dan dapat dijelaskan. Proyek ini tidak memuat data pribadi maupun kunci rahasia (`.gitignore` mengecualikan `data/*.json` dan `.env`).

9.5 Pengembangan Lanjutan

- Counter ID persisten agar ID tidak pernah dipakai ulang.
- Perintah `edit`, prioritas tugas, dan pencarian.
- Antarmuka web sederhana atau ekspor ke CSV.
- Tes cakupan (coverage) dan lint otomatis di GitHub Actions (CI dasar sudah disiapkan).

10. Penutup dan Lampiran

10.1 Kesimpulan

Proyek ini menunjukkan bahwa AI generatif sangat efektif mempercepat pemrograman, tetapi nilai terbesar muncul dari **proses manusia di sekitarnya**: merancang kebutuhan, menyusun prompt yang spesifik, mencurigai hasil, menguji dengan kasus tepi, mendiagnosis akar masalah, dan memperbaiki secara terukur. Draf pertama AI gagal pada 7 dari 10 tes; setelah tiga putaran evaluasi dan perbaikan, aplikasi lulus 19 dari 19 tes dan siap dijalankan oleh siapa pun lewat repositori GitHub.

10.2 Lampiran A - Struktur Repositori

```
task-manager-js/
|-- bin/tm.js                 titik masuk CLI
|-- src/                      cli, taskManager, validators, storage, config
|-- config/default.json       konfigurasi bawaan
|-- data/                     lokasi tasks.json (tidak di-commit)
|-- test/                     tes unit dan end-to-end (19 tes)
|-- docs/                     proposal, gambar, bukti pengujian, skrip
gambar
|-- .github/workflows/ci.yml  CI GitHub Actions
|-- README.md  LICENSE  package.json  .gitignore
```

10.3 Lampiran B - Perintah Git untuk Mengunggah ke GitHub

```
git remote add origin https://github.com/<username>/task-manager-js.git
git push -u origin main
git log --oneline           # menampilkan riwayat iterasi v1 -> v3
```

10.4 Lampiran C - Bukti Pengujian

Keluaran mentah tiap iterasi tersimpan di folder `docs/evidence/`: `test_v0_config_error.txt` (kegagalan skrip tes), `test_v1.txt` (7 gagal), `test_v2.txt` (10 lulus), `test_v2_cli.txt` (1 gagal ditemukan tes CLI), dan `test_v3_final.txt` (19 lulus).